

28 June 84, 1 April 86

This note will give some explanation on how to use Debug under MS/PC DOS with INTEL.HEX files. It will explain how to create an INTEL.HEX file with Ghex, our utility program, and how to change data in the file without creating a problem with the checksums. See the last section to modify Intel Hex files for even paragraph (16 byte) boundaries offset.

You can generate an INTEL.HEX file in several different ways. Under MS/PC DOS an assembler will usually create a .OBJ file and after it's linked you will have a .EXE or .COM file. A cross assembler, which can assemble source code for a processor other than the processor the assembler is running on can usually generate an INTEL.HEX file in addition to other types of files.

Examples to follow are for an eprom programmer like the 7128, 7228, 7956 or the 9000. You must have an INTEL.HEX file to start with, and we will make one if you don't have one. You must have DEBUG, PGMX, and GHEX on the same disk (to make explanation easier).

Using DEBUG, let's create a BINARY file to use GHEX on. In the following, the "A>" indicates the prompt your computer gave you. It may be "C>" on your computer, it doesn't make any difference, just make sure that the programs you need are on the same drive, or that you know how to "address" programs that are on a drive different from the one that you are logged onto. Also, a "<CR>" means that you are to strike the "return" key. A ";" means that a comment for your information follows and you are NOT to type that in too. Use the DOS manual to find the commands that are used for DEBUG to follow along with what's going on. In my book it's around chapter 12.

Type:

```
A>debug<CR>           ;enter debug.
-                     ;You get a "-" prompt and a cursor flashing.
-f100,10ff,ff<CR>      ;fill memory from 0100 to 10ff (1000h bytes)
-ntstfl.img<CR>        ;fill the file control block with tstfl.IMG
-r cx<CR>              ;Tell cx register how many bytes to write
:1000<CR>              ;1000h (4096) bytes
-w<CR>                 ;write 1kh bytes to file "tstfl.img"
-Q<CR>                 ;Quit to the system (DOS)
A>
```

If you have a file on the disk already called tstfl.img, it will be overwritten. When we "GHEX" the file, GHEX will check to see if there is a file called tstfl.hex on the disk, and will not let you proceed until you rename tstfl.hex to something else or erase it. The following will take tstfl.img and create a new file called tstfl.hex on the disk with record addresses from 0000h to 0fffh, since we are not specifying any offset. If we want an offset we can specify it as in the second example.

Type:

(first example)

```
A>GHEX TSTFL.IMG       ;Creates TSTFL.HEX on the disk.
```

(second example)

A>GHEX TSTFL.IMG 100 ;Create file, records fm 0100h to 1100h

You can also get an INTEL.HEX file on the disk by reading one in from the programmer. Select a 2732 eprom with the menu command (for a 1kh file), or use the option line with PGMX. Don't put any eprom in the programming socket.

Type:

A>PGMX TSTFL [R,MC]<CR> ;PGMX reads socket to INTEL.HEX file.

OR

<2732>^F

ENTER COMMAND -->TSTFL[R,MC<CR>

In either case the result will be a file by the filename TSTFL.HEX on the disk containing the contents of the eprom in the INTEL.HEX format with each record being an even 16d (10h) bytes long. We are now ready to operate.

The INTEL.HEX format is explained in your programmer manual. A quick recap of it follows eg. (spaces for clarity only!):

```
: 10 0000 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00<CR><lf>
1 2 3 4 5 < 16d (10h) data bytes > 6 checksum^ 7 8
# ascii bytes
```

1- programmer recognizes colon as a command that	
an INTEL.HEX record is to follow.	1
2- number of data bytes in this record 0 to 255d (00 to FFh)	2
3- load address of this record.	4
4- Data type byte (nearly always 00).	2
5- data bytes, in this case there are 16 BINARY data bytes	32
6- checksum	2
7- carriage return (required for GTEK programmers)	1
8- line feed	1
	"
	total ascii bytes =45

example file output by GHEX exactly 4096d (1000h) bytes long:

```
:10000000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF00
:10001000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF00
```

(more records here... 20h-0FDFh... shortened for brevity)

```
:100FE000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF11
:100FF000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF01
:00000000000
```

This makes loading a hex file to memory very easy. DEBUG will accept an INTEL.HEX file directly and load it to the memory addresses specified in the file. BE EXTREMELY CAREFUL about overwriting active areas of your memory if you have any explicit segment addresses. You will probably have to load that type of file directly to memory with DEBUG. Neither GHEX nor any of the programmers will create any segmented records for you to worry about.

Now we're ready to see an example or two about making changes in

an Intel HEX file without upsetting any of the checksums. The trick is to change the (ascii) hex file to binary in memory with DEBUG and operating on it there. We can then save the modified memory (binary code) to the disk and regenerate the INTEL.HEX (ascii) type file with GHEX. To start, we are going to enter the DEBUG command state and do everything from there. See the examples below as we talk about it. Give DEBUG a filename with the "n" command. Now we want to "load" it to DEBUG's default free memory area. You should assume that the .HEX file will start at 0000h. This means that it will overwrite the file control area (seg:0000h-seg:00ffh). We will ignore the segment for the time being. It will change depending on how much memory you have available, if you have any interrupts running and a lot of other factors. What you should know about it is that DEBUG assumes that all memory from that segment:0000h up is free memory. You probably won't have to worry about it unless you have files for eproms larger than 27512's, or if you are using the INTEL EXTENDED HEX format files (16 bit). See the section on Debug in your PC-DOS manual.

We issue the command L1000 to load the file to the load address (+1000h) specified at the beginning of each hex record. We could have given an offset of 100h, but all you want to do is use some offset of at least 100h to keep from overwriting any of the file control blocks below that address.

Now, if DEBUG didn't give you any error messages, you can proceed to modify memory, otherwise you can look it up in the DOS manual under the chapter about error messages. There is one whole chapter in the DOS manual about error messages issued by Debug, Edlin, etc. Refer to that chapter to interpret error messages.

You may do an "r" command to check to see what the size of your file is. The CX register will contain the number of bytes loaded to memory (if less than 65k) and BX will take the overflow (if over 65k). In our case the CX register will say 2000 and the BX register will have 0000. We will have to change the CX register to reflect that we really only have a 1kh file and then later, after we have modified our data, we will write from our offset address. Be sure to modify the CX register if it doesn't contain the proper number of bytes to write back to the disk.

Type:

```
A>DEBUG<CR>                ;ENTER DEBUG
-NTSTFL.HEX<CR>            ;ENTER FILENAME TO LOAD
-L1000<CR>                 ;LOAD FILE WITH OFFSET OF 1000H
-R<CR>                     ;LOOK AT CX IN PARTICULAR
<DISPLAY OF ALL THE REGISTERS HERE, LOOK AT CX IN PARTICULAR>
CX 2000                    ;FILE SIZE HERE .... PLUS OFFSET
```

Now for an example, let's modify the addresses at 0AEFh IN THE EPROM. Now you might see why I chose to add an offset of 1000h. It makes finding the right address to modify easier. I know that the address I want to modify is 1000h + 0AEFh = 1AEFh. If I had chosen an offset of 100h, which is the first available address (and the address that the "W" command uses for a default), I would have to add 0100h + 0AEFh = 0BEFh. It's just easier to me to use an offset where I don't really have to add anything

together. Do what you are comfortable with or have "memory" for.

Note that <SP> means press the space bar like <CR> means press the carriage return key. When you use the "E" command, you must be extremely careful not to give DEBUG an address that you didn't mean to. Check your offset carefully. After you issue the Eaddr<CR> command, it will display the memory location and the contents. If you don't want to modify that location, simply press the space bar and you will advance to the next location without modification. If you do want to modify, enter the data and then press the space bar. You will see the next data displayed. When you are done, press the <CR> or <return> key after you press the space bar for the last time. When you are done with the modifications, you are ready to save the data to the disk. You can also look at the data with the "d" command. See example below.

Type (continuing from the previous page):

```
-E1AE0<SP>.FF<SP>.FF<SP>.FF<SP>.FF<SP><CR> ;we didn't modify
-E1AEF<SP>.FF 41<SP>.FF 42<SP>.FF 43<SP><CR> ;we modified data
-D1AE0,AFF<CR>
SEGM:1AE0 FFFF FFFF FFFF FFFF FFFF FFFF FFFF FF41 .....A
SEGM:1AF0 4243 FFFF FFFF FFFF FFFF FFFF FFFF FFFF BC.....
-R CX<CR> ;Change the CX register to reflect 1kh bytes
:1000<CR>
-NTSTFL.bin<CR> ;give DEBUG a new file name to write to.
-W1000<CR> ;write 1kh bytes from address 1000h to TSTFL.bin
WRITING 1000H BYTES
-Q<CR> ;Quit DEBUG to go back to the DOS.
```

In the above example, after you check your data, change the CX register (if you haven't already) to reflect the actual number of binary hex bytes that you need to write to the disk. Give DEBUG the name of the file you want to write to. Usually you want to use a new name to be able to generate the new .HEX file with GHEX without having to rename or erase the old file. Use this method as a way of generating a "backup" of the old file. (Don't write over the old one!) Give the Write command now with the offset that you specified when you loaded the file originally. If you used an offset of 100h you don't have to tell DEBUG that. If it's 0ffh or 101h etc, you will have to be explicit. Please make a note that if you load a file like an EXE file, that the CS: register will be set to something other than what DEBUG is using. THIS IS DETERMINED ONLY BY THE EXTENSION OF THE FILENAME! If the extension is something other than .EXE or .HEX the CS: register will probably be the one that DEBUG is using. DEBUG writes data from where the CS: register is pointing, offset 0100H.

Now you should have a BINARY image of the original hex file, modified and on the disk with the new file name that you specified just before you wrote the file from DEBUG. We need to use INTEL.HEX (ascii) files with the programmer however, so use GHEX to generate the new .HEX file. If you need an offset from 0000h, be sure to specify it now. Remember that GHEX does not "Relocate" code for you. If you offset your code from some absolute address in the eprom, then it may not work after you get through with it. If it's a data type eprom,

like you would use with a pattern or mill machine, then you're probably safe. If it's program eprom data you just offset, then you're probably in trouble.

1- Convert to INTEL.HEX file: A>GHEX TESTA.DAT 1000<CR>

This converts our file to an INTEL.HEX file starting with a load address of 1000h. This is not significant to this 2732 eprom since it's highest address is 0FFFh. The data will "wrap-around" if you send it to the programmer. Address 0000h will be the same as 1000h in other words.

2- Load the file using DEBUG: A>DEBUG testa.HEX<CR>

We don't have to worry about overwriting the data in the addresses below 100h because we know that this file's addresses start at 1000h. If you had one that starts at 0000h you would use the "N" command and then "L100" it to memory with that offset. If you read in a file from the programmer then you will HAVE to use the offset method since the addresses will start at 0000h.

3- If we want to change a few bytes at address 1AB9h we then would issue the command: E1AB9<SP>. This will cause DEBUG to begin entering at that address. Type the data that you wish to be there. As an example let's enter the hex data that will display the word "HELLO" if you used Dump to display the hex and ascii data.

Type:

```
-E1ab9<SP> .FF 48<SP> .FF 45<SP> .FF 4B<SP> .FF 4B<SP> .FF 4F<SP><CR>
D1AB0,1ABF<CR>
```

```
1AB0 FF FF FF FF FF FF FF FF 48 45 4B 4B 4F FF FF .....HELLO..
- _
```

4- Now we need to save this changed information to the disk if we want to use it.

Type:

```
R CX<CR>
:1000<CR>
NNEWFILE.BIN<CR>
W1000<CR>
```

5- Use GHEX to convert back to a hex file we can send to the programmer.

Type:

```
A>GHEX NEWFILE.BIN<CR>
```

6- We should now have a file called NEWFILE.HEX on the disk, created by GHEX. We can send this file to the eprom programmer in this form.

```
A>PGMX NEWFILE [options]<CR>
```

The programmer will handle it from there. If you inserted and selected the right eprom of course. It will probably let you

know if you didn't. See the manual for instructions on using PGMX.

You may also get a binary file by using PGMX:

```
A>PGMX NEWFILE.IMG [%<CR>
```

Reads the selected eprom's data into the disk file newfile.img. The file size will exactly the size of the eprom, eg. 2732 = 4096 bytes, etc. If you also specify boundaries with the "@" command, you will get a file the size of what you specified, from the specified start address to the specified end address.

Some summary...

DEBUG commands:

Nfilespec.ext load file control block with your filespec.ext

L Read a file specified by the N command starting at 100h or in the case of an INTEL.HEX file, at the specified load addresses.

L100 Read a specified file into memory with a 100h offset. If it's an INTEL.HEX file starting at F00h, 100h will be added to all addresses and will go to memory starting at 1000h. (F00h+100h)

Eaaaa Enter data starting at address aaaah, up to the address where you enter a <CR> as data.

Fssss,eeee,dd Fill memory starting at address ssssh up to and including address eeeeh with the data ddh.

See the instruction manual for the model 9000 programmer or a file called READ.ME for instructions on using GHEX and PGMX.

You can offset data in an Intel Hex file by specifying a Segment record just before the data you wish to offset. The programmer adds this offset to the address counter inside the programmer, so the apparent addresses that are being sent to the programmer appear the same, but are actually offset by the USBA sent by the segment record. Refer to the section on the Intel Hex format in the manual for an example of the format.

Here are some segment records you could add to your code to offset your data within the eprom without otherwise modifying the file.

```
:02 0000 02 0000 FC ;usba record with segment set to 0000H
  ^  ^      ^  ^  ^---;checksum
  |  |      |  |-----;usba 0000H - FFFFH segment
  |  |      |-----;record type, 02 for USBA
  |  |-----;load address- always 0
  |-----;always 2 for a Segment record
```

```
:0200000020001FB ; 10H offset
```

```

:0200000020002FA      ; 20H
:0200000020003F9      ; 30H
:0200000020004F8      ; 40H
:0200000020005F7      ; 50H
:0200000020006F6      ; 60H
:0200000020007F5      ; 70H
:0200000020008F4      ; 80H
:0200000020009F3      ; 90H
:020000002000AF2      ; A0H
:020000002000BF1      ; B0H
:020000002000CF0      ; C0H
:020000002000DEF      ; D0H
:020000002000EEE      ; E0H
:020000002000FED      ; F0H
:0200000020010EC      ; 100H  OFFSET
:0200000020011EB      ; 110H
:0200000020020DC      ; 200H
:0200000020040BC      ; 400H
:02000000200807C      ; 800H
:0200000020100FB      ; 1000H
:0200000020200FA      ; 2000H
:0200000020400F8      ; 4000H
:0200000020800F4      ; 8000H

```

EXAMPLES

OFFSET A50H WITHIN AN EPROM:

```
:02000000200A557 <-----ADDED IN YOUR HEX FILE
:1000000007F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F00
```

OFFSET 110H WITHIN AN EPROM (CODE ALREADY STARTS AT 10H):

```
:0200000020010EC    <---- ADDED  
:100010007F7F7F7F7F7F7F7F7F7F7F7F7F7F7FF0  
:100020007F7F7F7F7F7F7F7F7F7F7F7F7F7FE0  
:100030007F7F7F7F7F7F7F7F7F7F7F7F7FD0  
(NEXT ADDRESS IS ACTUALLY 140H)
```

```
:0200000020009F3 (OFFSET 90H, NEXT LOAD ADDRESS IS 90H+40H=D0H)
:100040007F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7FC0
:100050007F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7FB0
:100060007F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7FA0
:100070007F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F90
:000000000000
```

[illegible]